

datalab 报告

姓名：迟子祎

学号：2025201685

test 截图：

```
● luna@LAPTOP-SCM975J8:~/datalab2026$ make
./btest
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
1      1      0      bitAnd
1      1      0      bitXor
2      2      0      byteSwap
3      3      0      logicalShift
4      4      0      float_i2f
4      4      0      leftBitCount
4      4      0      floatPower2
4      4      0      floatScale2
3      3      0      reverse
2      2      0      sameSign
4      4      0      logtwo
3      3      0      float64_f2i
```

解题报告

亮点

- 1. float_i2f
- 2. reverse
- 3. logtwo
- 4. byteSwap
- 5. leftBitCount
- 6. floatScale2
- 7. floatPower2
- 8. float64_f2i
- 9. logicalShift
- 10. bitXor

讲解题目思路

bitXAnd

思路：

利用 $\sim (x \& y) = \sim x | \sim y$ 推导得到

```
int bitAnd(int x, int y) {
    return ~(~x|~y);
}
```

bitXor

思路:

同理bitxand得到xy或, 用 $\sim (x \& y)$ 排除1 1为1的情况

```
// 附上题目解题代码
int bitXor(int x, int y) {
    return ~( (~x)&(~y) ) & ~(x&y));
}
```

samesign

思路:

如果x是0, y是0 same

如果x是0, y不是0 not same

如果x不是0, y是0, not same

如果x不是0, y不是0 返回sign的异或值取反(相同为1不同为0)

```
// 附上题目解题代码
int samesign(int x, int y) {
    if(!x)//如果x是0
    {
        if(!y)//y也是0 相同1
        {return 1;}
        else//y不是0 不同0
        {return 0;}
    }
    else
    {
        if(!y)//如果y是0, 不同0
        {return 0;}
        else
        {
            return !((x >> 31) ^ (y >> 31));
        }
    }
    return 0;
}
```

logtwo

思路:

二分查找最高位1位置

```
// 附上题目解题代码
int logtwo(int v) {
    int result=0;
    int step;

    step=((v>>16)>0)<<4;
```

```

    result=result|step;
    v=v>>step;

    step=((v>>8)>0)<<3;
    result=result|step;
    v=v>>step;

    step=((v>>4)>0)<<2;
    result=result|step;
    v=v>>step;

    step=((v>>2)>0)<<1;
    result=result|step;
    v=v>>step;

    result = result | ((v >> 1) > 0);

    return result;
}

```

byteSwap

思路:

交换两个字节的位置

先计算两个字节的位置，提取两个字节，把对应位置清零，再用位运算移到交换后的位置，|写入

```

// 附上题目解题代码
int byteSwap(int x, int n, int m) {
    // 计算两个字节的位置
    int shiftN = n << 3;
    int shiftM = m << 3;

    // 提取两个字节
    int bytem = (x >> shiftM) & 0xFF;
    int byten = (x >> shiftN) & 0xFF;

    // 把 x 对应的两个字节清零
    int clear = x & ~((0xFF << shiftN) | (0xFF << shiftM));

    // 交换位置
    int newm = byten << shiftM;
    int newn = bytem << shiftN;

    // 合并
    return newm | newn | clear;
}

```

reverse

思路:

类似归并排序，先相邻1个交换，再相邻2个，四个，八个，十六个，然后返回

```

unsigned reverse(unsigned v) {
    v = ((v & 0xAAAAAAAAU)>>1) | ((v & 0x55555555U)<<1); //1010 0101 1
    v = ((v & 0xCCCCCCCCU)>>2) | ((v & 0x33333333U)<<2); //1100 0011 2
    v = ((v & 0xF0F0F0F0U)>>4) | ((v & 0x0F0F0F0FU)<<4); //1111 0000 0000 1111 4
    v = ((v & 0xFF00FF00U)>>8) | ((v & 0x00FF00FFU)<<8); //11111111 00000000 8
    v = ((v & 0xFFFF0000U)>>16) | ((v & 0x0000FFFFU)<<16);
    return v;
}

```

logicalShift

思路:

x向左移动n位，用掩码提取后面几位，去除掉有符号带来的1

```
// 附上题目解题代码
int logicalShift(int x, int n) {
    return x>>n & 0xFFFFFFFFU>>n; //x向左移动n位，用掩码提取后面几位，去除掉有符号带来的1
}
```

leftBitCount

思路:

利用1, 2, 4, 8, 16可组成31一下任何数，计算左连续1个数

```
// 附上题目解题代码
int leftBitCount(int x) {
    int y = ~x;
    int n = 0;
    int f;

    f = !(y & 0xFFFF0000);
    n = n + (f << 4);
    y = y << (f << 4);

    f = !(y & 0xFF000000);
    n = n + (f << 3);
    y = y << (f << 3);

    f = !(y & 0xF0000000);
    n = n + (f << 2);
    y = y << (f << 2);

    f = !(y & 0xC0000000);
    n = n + (f << 1);
    y = y << (f << 1);

    f = !(y & 0x80000000);
    n = n + f;
    y = y << f;

    n = n + !y; // 处理 x = -1, 即 ~x = 0 的情况，前导零应为 32
    return n;
}
```

float_i2f

思路:

如果是0或者最小负数，单独处理

保留符号得到sign，用计数器记录数位得到exp，如果大于23时frac考虑舍位问题，不大于直接存即可

```
// 附上题目解题代码
unsigned float_i2f(int x) {
    if(x == 0)
    {
        return 0;
    }
    if(x == 0x80000000)
    {
        return 0xCF000000;
    }
}
```

```

}
unsigned sign = (x >> 31) & 1; // 仅保留bit31符号位，其余为0
sign = sign << 31;
//移码
unsigned abs_x;
if(x < 0)
{
    abs_x = ~x + 1;
}
else
{
    abs_x = x;
}
int msb=0;
unsigned temp=abs_x;
while(temp>>1)
{
    temp=temp>>1;
    msb++;
}
unsigned exp= (msb+127)<<23;
//比23位小的时候
unsigned frac;
if(msb<=23)
{
    frac = (abs_x & ((1<<msb)-1)) << (23-msb);
}
else
{
    int shift = msb-23; //右移位数
    frac=abs_x>>(shift);
    frac &= 0x7FFFFFFF; //去掉第一位
    //舍入
    int round = abs_x & ((1<<shift)-1);
    unsigned half = 1 << (shift-1);
    if(round > half || (round == half && (frac & 1)))
    {
        frac++;
        if(frac == 0x800000)
        {
            frac = 0;
            exp += (1 << 23);
        }
    }
}
return sign | exp | frac;
}

```

floatScale2

思路:

上来先分割符号位，移码，尾数

如果是0或者无穷（NaN）直接返回

如果是非规格化数，直接左移一位（已经考虑规格化溢出问题）

如果是规格化数，指数+1

如果溢出位无穷，尾数是0

```

unsigned floatScale2(unsigned uf)
{
    unsigned sign = uf & 0x80000000; //取最高位1-注意是8
    unsigned exp = (uf >> 23) & 0x000000FF;

```

```

    unsigned frac = uf & 0x007FFFFF; //取后23位-注意是7

    /* 0 */
    if (!uf)
    {
        return uf;
    }

    /* NaN 或无穷 */
    if (exp == 255)
    {
        return uf;
    }

    /* 非规格化数 */
    if (exp == 0)
    {
        frac = frac << 1;

        return sign | frac;
    }

    /* 规格化数 */
    exp = exp + 1; //乘以2是指数加一

    /* 规格化数乘 2 后溢出为无穷 */
    if (exp == 255) //如果exp溢出了 1 0000 0000
    {
        frac = 0;
    }

    return sign | (exp << 23) | frac;
}

```

float64_f2i

```

int float64_f2i(unsigned uf1, unsigned uf2)
{
    /*
     * double:
     * sign:    uf2 bit31
     * exp:     uf2 bit30~20
     * fraction:
     *         uf2 bit19~0 + uf1 bit31~0
     */

    unsigned sign;
    unsigned exp;
    unsigned frac_high;
    unsigned frac_low;

    sign = uf2 >> 31;

    exp = (uf2 >> 20) & 0x7FF;

    frac_high = uf2 & 0x000FFFFF;

    frac_low = uf1;

    /*
     * 特殊情况1:
     * exp == 0 表示非规格化数
     * 非常小, 转换int一定为0
     */
    if (exp == 0)

```

```

{
    return 0;
}

/*
 * 得到真实指数
 * double bias = 1023
 */
int E = exp-1023;

/*
 * 情况2:
 * 小于1
 */
if (E < 0)
{
    return 0;
}

/*
 * 情况3:
 * 判断溢出
 *
 * int最大31位有效
 *
 * 注意:
 * E > 31 一定溢出
 * E == 31 还要看符号和有效数字
 */
if (E > 31)
{
    return 0x80000000;
}

/*
 * 构造53位有效数字:
 *
 * M = 1.fraction
 *
 * 注意:
 * 这里不能真的放进unsigned
 * 因为53位 > 32位
 *
 * 所以需要根据E判断:
 *
 * E <= 20:
 *     只需要uf2里的隐藏位+高fraction
 *
 * E > 20:
 *     需要结合uf1
 */

unsigned result;

if (E <= 20)
{
    /*
     * 构造:
     *
     * 1 + fraction_high
     *
     * 然后右移(20-E)
     *
     */
    result = ((1<<20)|frac_high)>>(20-E);
}
else

```

```

{
    /* E > 20:
    * hidden bit
    * fraction_high
    * fraction_low
    * 组合成整数部分 */

    unsigned high;

    /*
    * high:
    * 1.fraction_high
    * 共21位
    */
    high = (1 << 20) | frac_high;

    /*
    * E-20:
    * 因为high已经包含了隐藏位+20位fraction
    * 如果E=21:
    * 还需要向左移动1位
    */
    int shift = E - 20;

    if (shift < 32)
    {
        /*
        * high左移贡献高位
        * frac_low补充低位
        */
        result = (high << shift)
            | (frac_low >> (32 - shift));
    }
    else
    {
        /*
        * shift>=32:
        * fraction_low已经全部进入整数
        */
        result = high << shift;
    }
}

/*
* 符号处理
*/
if (sign)
{
    result = -result;
}

return result;
}

```

floatPower2

思路:

sign永远是0

如果太大，返回无穷

如果是规格化数，移码 $x+127$

如果是非规格化数1，计算1在frac中的位置

如果太小，返回0

```
unsigned floatPower2(int x) {
    unsigned sign = 0;
    unsigned exp;
    unsigned frac;

    /*情况1: 太大, 超过float表示范围*/
    if (x > 128)
    {
        // 返回 +INF
        return 0x7f800000;
    }

    /*情况2: 规格化数*/
    else if (x >= -126)
    {
        // exponent = x + 偏置
        exp= x + 127;
        // fraction = 0
        frac=0;
        // 返回 sign | (exp << 23) | frac
        return sign|(exp<<23)|frac;
    }

    /*情况3: 非规格化数*/
    else if (x >= -149)
    {
        // exponent = 0
        exp=0;
        // 根据 x 计算 fraction 中1的位置
        frac= 1<<(x+149);
        // 返回 sign | frac
        return sign|frac;
    }

    /*
     * 情况4: 太小, 连非规格化都表示不了
     */
    else
    {
        // 返回0
        return 0;
    }
}
```

反馈/收获/感悟/总结

参考的重要资料

无